

Cloud Spanner：使用 Go 建立遊戲排行榜

來源：<https://codelabs.developers.google.com/codelabs/cloud-spanner-leaderboard-go?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將瞭解如何使用含有修訂時間戳記欄的 Cloud Spanner 資料庫資料表，建立遊戲排行榜。

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. 設定 Cloud Spanner 執行個體](#)
4. [4. 建立資料庫和結構定義](#)
5. [5. 載入資料](#)
6. [6. 執行排行榜查詢](#)
7. [7. 清除所用資源](#)
8. [8. 恭喜！](#)

1. 總覽

[Google Cloud Spanner](#) 是全代管的關聯式資料庫服務，可水平擴充、遍及全球，提供 ACID 交易和 SQL 語意，同時兼顧效能和高可用性。

在本實驗室中，您將學習如何設定 Cloud Spanner 執行個體。您將逐步建立可用於遊戲排行榜的資料庫和結構定義。首先，您要建立「Players」資料表來儲存球員資訊，以及「Scores」資料表來儲存球員得分。

接著，您會將範例資料填入資料表。接著，您將執行一些前十大範例查詢，最後刪除執行個體來釋出資源，完成實驗室。

課程內容

- 如何設定 Cloud Spanner 執行個體。
- 如何建立資料庫和資料表。
- 如何使用修訂時間戳記欄。
- 如何將資料載入 Cloud Spanner 資料庫資料表，並加上時間戳記。
- 如何查詢 Cloud Spanner 資料庫。
- 如何刪除 Cloud Spanner 執行個體。

所需物品

- 網路瀏覽器，例如 [Chrome](#) 或 [Firefox](#)。
-

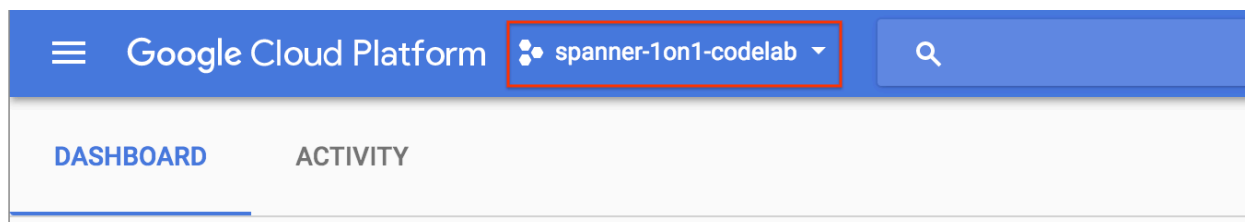
步驟 2 · 約 5 分鐘

2. 設定和需求

自修實驗室環境設定

如果您沒有 Google 帳戶 (Gmail 或 Google 應用程式)，請先[建立帳戶](#)。登入 Google Cloud Platform 主控台 (console.cloud.google.com)，然後建立新專案。

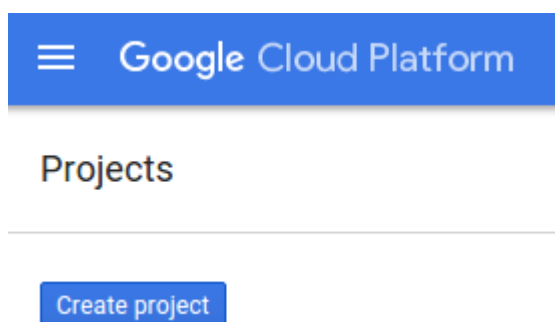
如果您已有專案，請按一下主控台左上方的專案選取下拉式選單：



然後在隨即顯示的對話方塊中，按一下「NEW PROJECT」(新專案) 按鈕，建立新專案：



如果您還沒有專案，應該會看到如下對話方塊，請建立第一個專案：



在隨後的專案建立對話方塊中，您可以輸入新專案的詳細資料：

New Project

Project name ?

spanner-1on1-codelab

Your project ID will be spanner-1on1-codelab ? [Edit](#)

Billing account ?

Choose account

Create

Cancel

請記住專案 ID，所有 Google Cloud 專案的專案 ID 都是不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

接下來，如果尚未啟用，請在開發人員控制台中[啟用帳單](#)，以便使用 Google Cloud 資源，並[啟用 Cloud Spanner API](#)。

Billing

This project has no billing account

This project is not linked to a billing account

[Link a billing account](#)

or

[Manage billing accounts](#)

完成本程式碼研究室的費用不應超過數美元，但如果您決定使用更多資源，或是將資源繼續執行 (請參閱本文件結尾的「清除」一節)，則可能會增加費用。Google Cloud Spanner 的定價說明文件請參閱[這裡](#)。

Google Cloud Platform 新使用者享有價值 **\$300 美元的免費試用期**，因此本程式碼研究室應完全免費。

Google Cloud Shell 設定

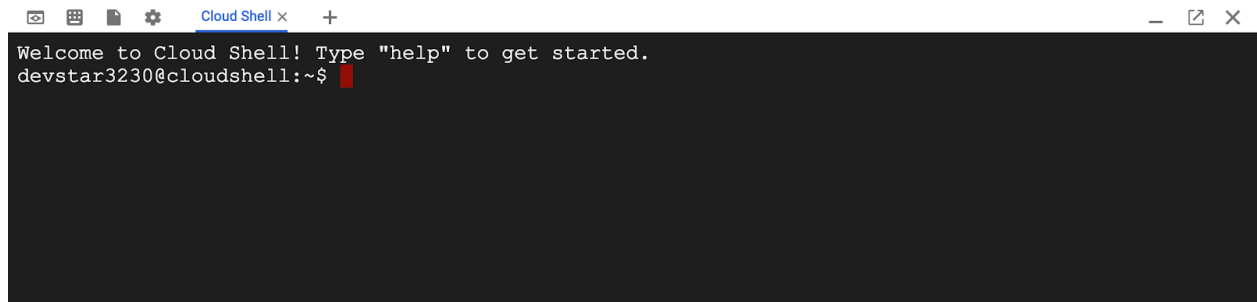
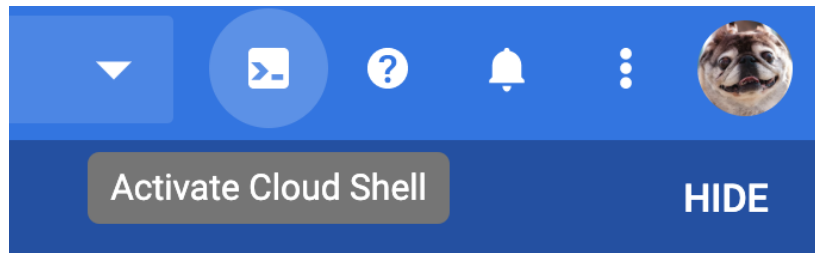
雖然可以透過筆電遠端操作 Google Cloud 和 Spanner，但在本程式碼研究室中，我們將使用 [Google Cloud Shell](#)，這是在雲端執行的指令列環境。

這部以 Debian 為基礎的虛擬機器，搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。也就是說，您只需要瀏覽器 (Chromebook 也可以) 就能完成本程式碼研究室。

1. 如要從 Cloud 控制台啟用 Cloud Shell，只要按一下「啟用 Cloud Shell」



即可 (佈建並連線至環境的作業需要一些時間才能完成)。



連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID `PROJECT_ID`。

```
gcloud auth list
```

指令輸出

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
```

注意：Cloud Shell 已預先安裝 `gcloud` 指令列工具，並支援 Tab 鍵完成功能，相信您一定會喜歡。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

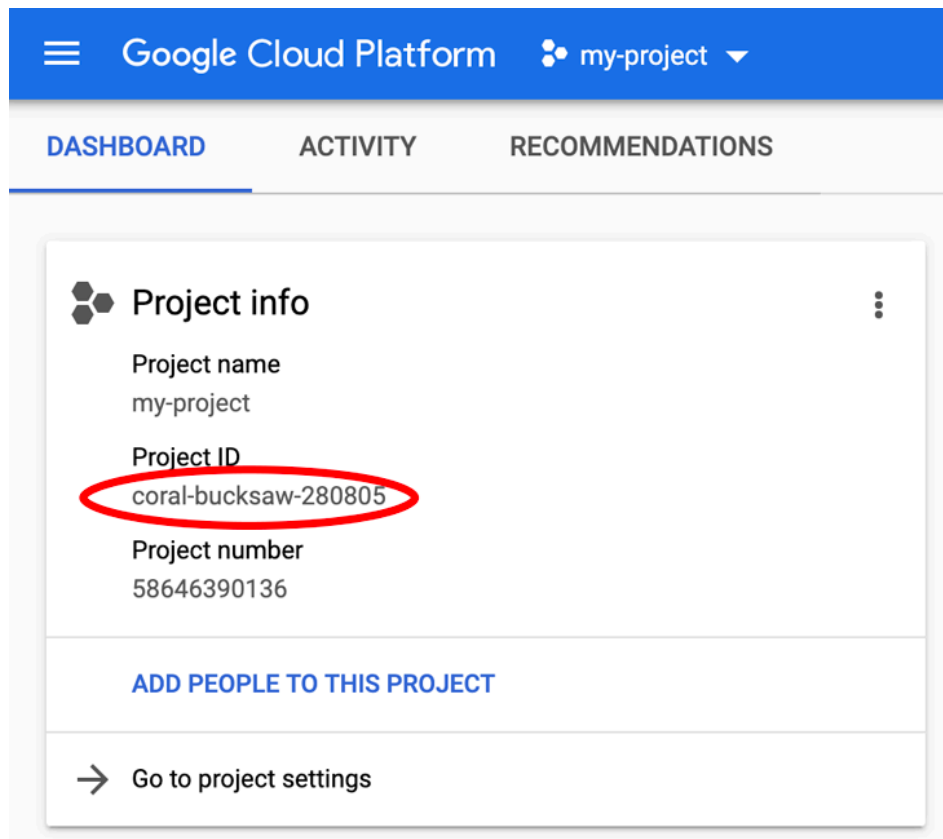
指令輸出

```
[core]
project = <PROJECT_ID>
```

如果專案未設定，請發出下列指令：

```
gcloud config set project <PROJECT_ID>
```

在尋找「`PROJECT_ID`」嗎？請檢查您在設定步驟中使用的 ID，或在 Cloud 控制台資訊主頁中尋找：



Cloud Shell 也會預設設定部分環境變數，這些變數在您執行後續指令時可能很有用。

```
echo $GOOGLE_CLOUD_PROJECT
```

指令輸出

```
<PROJECT_ID>
```

2. 最後，設定預設可用區和專案。

```
gcloud config set compute/zone us-central1-f
```

你可以選擇各種不同區域。詳情請參閱「[地區和區域](#)」。

注意：如果在自己的機器上執行 `gcloud`，各個工作階段都會保留這些設定。但在 Cloud Shell 中，每次啟動新工作階段或重新連線時，都必須重新設定。

摘要

在這個步驟中，您會設定環境。

下一步

接下來，您將設定 Cloud Spanner 執行個體。

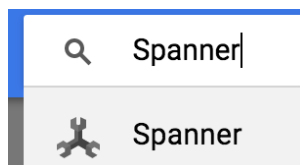
步驟 3 · 約 3 分鐘

3. 設定 Cloud Spanner 執行個體

在本步驟中，我們將為這個程式碼研究室設定 Cloud Spanner 執行個體。在左上方的「漢堡」選單中搜尋 Spanner 項目



，或按下「/」並輸入「Spanner」來搜尋



接著，按一下

+ CREATE INSTANCE

並填寫表單，輸入執行個體的執行個體名稱 **cloudspanner-leaderboard**，選擇設定 (選取區域執行個體)，並設定節點數量。在本程式碼研究室中，我們只需要 1 個節點。如要使用實際工作環境執行個體，並符合 Cloud Spanner 服務等級協議的資格，您需要在 Cloud Spanner 執行個體中執行 3 個以上的節點。

最後，按一下「建立」，幾秒內您就能使用 Cloud Spanner 執行個體。

Create an instance

Instance name

For display purposes only.

Instance ID

Unique identifier for instance. Permanent.

Configuration

Determines where your data and nodes are located. Affects cost, performance, and replication. This choice is permanent. Select a configuration to view its details.



Regional



Multi-region

Nodes

Add nodes to increase data throughput and queries per second (QPS). Affects billing.

 [Node guidance](#)

Cost

Storage cost depends on GB stored per month. Nodes cost is an hourly charge for the number of nodes in your instance. [Learn more](#)

Nodes cost

\$0.90 per hour

Storage cost

\$0.30 per GB/month

在下一個步驟中，我們將使用 Go 用戶端程式庫，在新執行個體中建立資料庫和結構定義。

步驟 4 · 約 10 分鐘

4. 建立資料庫和結構定義

在這個步驟中，我們要建立資料庫範例和結構定義。

我們將使用 Go 用戶端程式庫建立兩個資料表：一個是 Players 資料表，用於儲存玩家資訊；另一個是 Scores 資料表，用於儲存玩家分數。為此，我們將逐步說明如何在 Cloud Shell 中建立 Go 控制台應用程式。

首先，請在 Cloud Shell 中輸入下列指令，從 GitHub 複製本程式碼研究室的範例程式碼：

```
export GO111MODULE=auto
go get -u github.com/GoogleCloudPlatform/golang-samples/spanner/...
```

然後將目錄變更為「leaderboard」目錄，您將在其中建立應用程式。

```
cd gopath/src/github.com/GoogleCloudPlatform/golang-samples/spanner/spanner_leaderboard
```

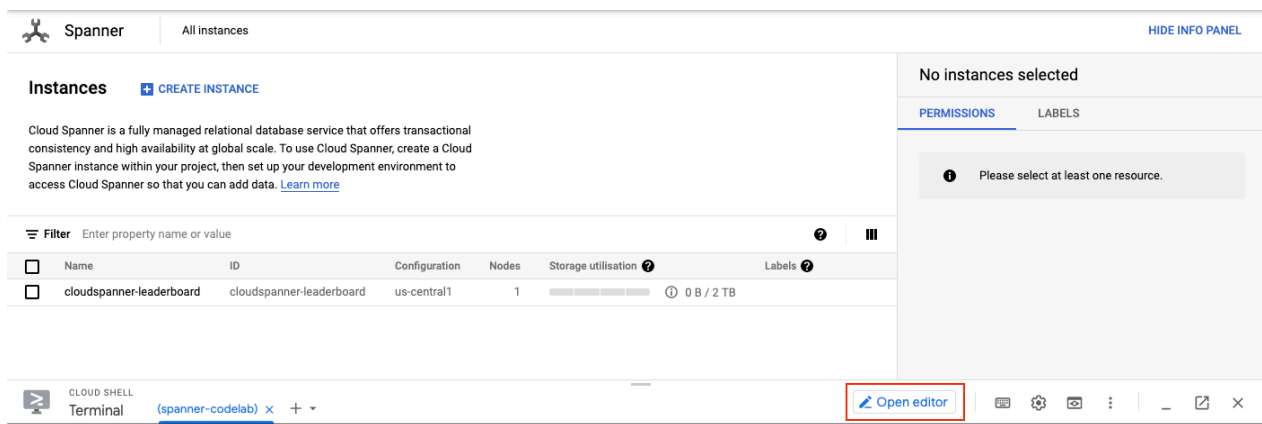
本程式碼研究室所需的所有程式碼都位於現有的 `golang-samples/spanner/spanner_leaderboard/` 目錄中，做為名為 `leaderboard` 的可執行 Go 應用程式，供您在完成本程式碼研究室時參考。我們會建立新目錄，並分階段建構排行榜應用程式的副本。

建立名為「codelab」的應用程式新目錄，然後使用下列指令將目錄變更為該目錄：

```
mkdir codelab && cd $_
```

現在，我們來建立名為「Leaderboard」的基本 Go 應用程式，該應用程式會使用 Spanner 用戶端程式庫建立排行榜，其中包含兩個資料表：「Players」和「Scores」。您可以在 Cloud Shell 編輯器中執行這項操作：

按一下下方醒目顯示的「開啟編輯器」圖示，開啟 Cloud Shell 編輯器：



在 `~/gopath/src/github.com/GoogleCloudPlatform/golang-samples/spanner/codelab` 資料夾中，建立名為「leaderboard.go」的檔案。

- 首先，請務必在 Cloud Shell 編輯器的資料夾清單中選取「codelab」資料夾。
- 然後在 Cloud Shell 編輯器的「File」選單中選取「New File」。
- 輸入「leaderboard.go」做為新檔案的名稱。

這是應用程式的主要檔案，其中包含應用程式程式碼和參照，可納入任何依附元件。

如要建立 `leaderboard` 資料庫和 `Players` 與 `Scores` 資料表，請複製 (Ctrl + P) 下列 Go 程式碼，然後貼到 `leaderboard.go` 檔案中 (Ctrl + V)：

```

package main

import (
    "context"
    "flag"
    "fmt"
    "io"
    "log"
    "os"
    "regexp"
    "time"

    "cloud.google.com/go/spanner"
    database "cloud.google.com/go/spanner/admin/database/apiv1"

    adminpb "google.golang.org/genproto/googleapis/spanner/admin/database/v1"
)

type adminCommand func(ctx context.Context, w io.Writer, adminClient
*database.DatabaseAdminClient, database string) error

func createDatabase(ctx context.Context, w io.Writer, adminClient
*database.DatabaseAdminClient, db string) error {
    matches := regexp.MustCompile("^(.*)/databases/(.*)$").FindStringSubmatch(db)
    if matches == nil || len(matches) != 3 {
        return fmt.Errorf("Invalid database id %s", db)
    }
    op, err := adminClient.CreateDatabase(ctx, &adminpb.CreateDatabaseRequest{
        Parent:      matches[1],
        CreateStatement: "CREATE DATABASE `" + matches[2] + "`",
        ExtraStatements: []string{
            `CREATE TABLE Players(
                PlayerId INT64 NOT NULL,
                PlayerName STRING(2048) NOT NULL
            ) PRIMARY KEY(PlayerId)`,
            `CREATE TABLE Scores(
                PlayerId INT64 NOT NULL,
                Score INT64 NOT NULL,
                Timestamp TIMESTAMP NOT NULL
                OPTIONS(allow_commit_timestamp=true)
            ) PRIMARY KEY(PlayerId, Timestamp),
            INTERLEAVE IN PARENT Players ON DELETE NO ACTION`,
        },
    })
    if err != nil {
        return err
    }
    if _, err := op.Wait(ctx); err != nil {
        return err
    }
    fmt.Fprintf(w, "Created database [%s]\n", db)
}

```

```

        return nil
    }

    func createClients(ctx context.Context, db string) (*database.DatabaseAdminClient,
    *spanner.Client) {
        adminClient, err := database.NewDatabaseAdminClient(ctx)
        if err != nil {
            log.Fatal(err)
        }

        dataClient, err := spanner.NewClient(ctx, db)
        if err != nil {
            log.Fatal(err)
        }

        return adminClient, dataClient
    }

    func run(ctx context.Context, adminClient *database.DatabaseAdminClient, dataClient
    *spanner.Client, w io.Writer,
    cmd string, db string, timespan int) error {
        // createdatabase command
        if cmd == "createdatabase" {
            err := createDatabase(ctx, w, adminClient, db)
            if err != nil {
                fmt.Fprintf(w, "%s failed with %v", cmd, err)
            }
            return err
        }
        return nil
    }

    func main() {
        flag.Usage = func() {
            fmt.Fprintf(os.Stderr, `Usage: leaderboard <command> <database_name>
[command_option]

    Command can be one of: createdatabase

Examples:
    leaderboard createdatabase projects/my-project/instances/my-
instance/databases/example-db
    - Create a sample Cloud Spanner database along with sample tables in your
project.
`)
        }

        flag.Parse()
        flagCount := len(flag.Args())
        if flagCount != 2 {
            flag.Usage()
            os.Exit(2)
        }
    }

```

```

    }

    cmd, db := flag.Arg(0), flag.Arg(1)
    // Set timespan to zero, as it's not currently being used
    var timespan int = 0

    ctx, cancel := context.WithTimeout(context.Background(), 1*time.Minute)
    defer cancel()
    adminClient, dataClient := createClients(ctx, db)
    if err := run(ctx, adminClient, dataClient, os.Stdout, cmd, db, timespan); err != nil
{
    os.Exit(1)
}
}

```

在 Cloud Shell 編輯器的「File」(檔案) 選單中選取「Save」(儲存)，即可儲存對 `leaderboard.go` 檔案所做的變更。

您可以使用 `golang-samples/spanner/spanner_leaderboard` 目錄中的 `leaderboard.go` 檔案，查看啟用 `createdatabase` 指令後 `leaderboard.go` 檔案的範例。

如要在 Cloud Shell 中建構應用程式，請從 `leaderboard.go` 檔案所在的 `codelab` 目錄執行「go build」：

```
go build leaderboard.go
```

應用程式建構完成後，請輸入下列指令，在 Cloud Shell 中執行產生的應用程式：

```
./leaderboard
```

您會看到如下所示的輸出：

```

Usage: leaderboard <command> <database_name> [command_option]

    Command can be one of: createdatabase

Examples:
    leaderboard createdatabase projects/my-project/instances/my-
instance/databases/example-db
    - Create a sample Cloud Spanner database along with sample tables in
your project.

```

從這個回覆中，我們可以看到這是 `Leaderboard` 應用程式，目前有一個可能的指令：`createdatabase`。我們可以發現 `createdatabase` 指令的預期引數是字串，其中包含特定執行個體 ID 和資料庫 ID。

現在請執行下列指令。**請務必**將 `my-project` 替換為您在本程式碼研究室一開始建立的專案 ID。

```
./leaderboard createdatabase projects/my-project/instances/cloudspanner-
leaderboard/databases/leaderboard
```

幾秒後，您應該會看到類似以下的回應：

Created database [projects/my-project/instances/cloudspanner-leaderboard/databases/leaderboard]

在 Cloud 控制台的「Cloud Spanner databases overview」部分，您應該會在左側選單中看到新的資料庫和資料表。

Spanner

DATABASE

Overview

Monitoring

Query stats

Import/Export

Backup/restore

Query

Release Notes

All instances > INSTANCE cloudspanner-leaderboard: Overview > DATABASE leaderboard: Overview

Overview

Database name	leaderboard
Encryption type	Google-managed
Version retention period	1 hour

CPU utilisation (mean)
7.75%

Operations
Read: -/s
Write: -/s

Throughput
Read: -/s
Write: -/s

Total database storage
0 B

Tables

+ CREATE TABLE

Filter Filter tables

Name ↑	Indexes	Interleaved in
Players	—	—
Scores	—	Players

SHOW EQUIVALENT DDL

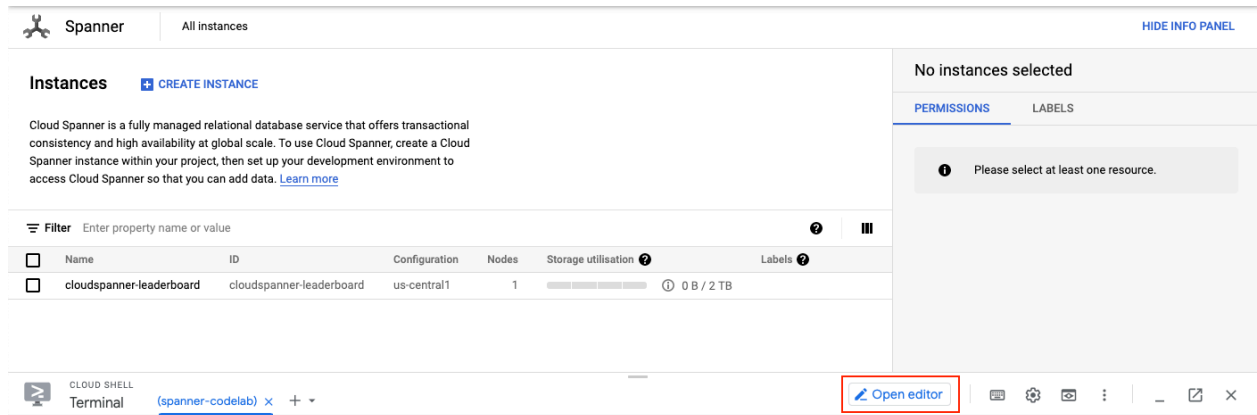
在下一個步驟中，我們會更新應用程式，將一些資料載入新的資料庫。

步驟 5 · 約 12 分鐘

5. 載入資料

現在我們有一個名為 `leaderboard` 的資料庫，其中包含兩個資料表：`Players` 和 `Scores`。現在，讓我們使用 Go 用戶端程式庫，在 `Players` 表格中填入玩家，並在 `Scores` 表格中填入每位玩家的隨機分數。

如果尚未開啟，請點選下方醒目顯示的圖示，開啟 Cloud Shell 編輯器：



接著，在 Cloud Shell 編輯器中編輯 `leaderboard.go` 檔案，新增可用於將 100 名玩家插入 `Players` 資料表的 `insertplayers` 指令。我們也會新增 `insertscores` 指令，可用於在 `Scores` 資料表中，為 `Players` 資料表中的每位玩家插入 4 個隨機分數。

首先，請更新 `leaderboard.go` 檔案頂端的 `imports` 區段，取代目前內容，完成後應如下所示：

```
import (  
    "context"  
    "flag"  
    "fmt"  
    "io"  
    "log"  
    "math/rand"  
    "os"  
    "regexp"  
    "time"  
  
    "cloud.google.com/go/spanner"  
    database "cloud.google.com/go/spanner/admin/database/apiv1"  
  
    "google.golang.org/api/iterator"  
    adminpb  "google.golang.org/genproto/googleapis/spanner/admin/database/v1"  
)
```

接著在檔案頂端，於以「`type adminCommand ...`」開頭的行下方，新增指令類型和指令清單，完成後應如下所示：

```
type adminCommand func(ctx context.Context, w io.Writer, adminClient
*database.DatabaseAdminClient, database string) error

type command func(ctx context.Context, w io.Writer, client *spanner.Client) error
var (
    commands = map[string]command{
        "insertplayers": insertPlayers,
        "insertscores":  insertScores,
    }
)
```

接著，在現有 `createdatabase()` 函式下方新增下列 `insertPlayers` 和 `insertScores` 函式：

```

func insertPlayers(ctx context.Context, w io.Writer, client *spanner.Client) error {
    // Get number of players to use as an incrementing value for each PlayerName to be
    inserted
    stmt := spanner.Statement{
        SQL: `SELECT Count(PlayerId) as PlayerCount FROM Players`,
    }
    iter := client.Single().Query(ctx, stmt)
    defer iter.Stop()
    row, err := iter.Next()
    if err != nil {
        return err
    }
    var numberOfPlayers int64 = 0
    if err := row.Columns(&numberOfPlayers); err != nil {
        return err
    }
    // Initialize values for random PlayerId
    rand.Seed(time.Now().UnixNano())
    min := 1000000000
    max := 9000000000
    // Insert 100 player records into the Players table
    _, err = client.ReadWriteTransaction(ctx, func(ctx context.Context, txn
*spanner.ReadWriteTransaction) error {
        stmts := []spanner.Statement{}
        for i := 1; i <= 100; i++ {
            numberOfPlayers++
            playerId := rand.Intn(max-min) + min
            playerName := fmt.Sprintf("Player %d", numberOfPlayers)
            stmts = append(stmts, spanner.Statement{
                SQL: `INSERT INTO Players
                    (PlayerId, PlayerName)
                    VALUES (@playerID, @playerName)`,
                Params: map[string]interface{}{
                    "playerID":  playerId,
                    "playerName": playerName,
                },
            })
        }
        _, err := txn.BatchUpdate(ctx, stmts)
        if err != nil {
            return err
        }
        return nil
    })
    fmt.Fprintf(w, "Inserted players \n")
    return nil
}

func insertScores(ctx context.Context, w io.Writer, client *spanner.Client) error {
    playerRecordsFound := false
    // Create slice for insert statements
    stmts := []spanner.Statement{}

```

```

// Select all player records
stmt := spanner.Statement{SQL: `SELECT PlayerId FROM Players`}
iter := client.Single().Query(ctx, stmt)
defer iter.Stop()
// Insert 4 score records into the Scores table for each player in the Players table
for {
    row, err := iter.Next()
    if err == iterator.Done {
        break
    }
    if err != nil {
        return err
    }
    playerRecordsFound = true
    var playerID int64
    if err := row.ColumnByName("PlayerId", &playerID); err != nil {
        return err
    }
    // Initialize values for random score and date
    rand.Seed(time.Now().UnixNano())
    min := 1000
    max := 1000000
    for i := 0; i < 4; i++ {
        // Generate random score between 1,000 and 1,000,000
        score := rand.Intn(max-min) + min
        // Generate random day within the past two years
        now := time.Now()
        endDate := now.Unix()
        past := now.AddDate(0, -24, 0)
        startDate := past.Unix()
        randomDateInSeconds := rand.Int63n(endDate-startDate) + startDate
        randomDate := time.Unix(randomDateInSeconds, 0)
        // Add insert statement to stmts slice
        stmts = append(stmts, spanner.Statement{
            SQL: `INSERT INTO Scores
                (PlayerId, Score, Timestamp)
                VALUES (@playerID, @score, @timestamp)`,
            Params: map[string]interface{}{
                "playerID": playerID,
                "score":     score,
                "timestamp": randomDate,
            },
        })
    }
}

if !playerRecordsFound {
    fmt.Fprintln(w, "No player records currently exist. First insert players then
insert scores.")
} else {
    _, err := client.ReadWriteTransaction(ctx, func(ctx context.Context, txn
*spanner.ReadWriteTransaction) error {

```

```

        // Commit insert statements for all scores to be inserted as a single
transaction
        _, err := txn.BatchUpdate(ctx, stmts)
        return err
    })
    if err != nil {
        return err
    }
    fmt.Fprintln(w, "Inserted scores")
}
return nil
}

```

然後，如要讓 `insert` 指令正常運作，請在應用程式的「run」函式中，於 `createdatabase` 處理陳述式下方加入下列程式碼，並取代 `return nil` 陳述式：

```

// insert and query commands
cmdFn := commands[cmd]
if cmdFn == nil {
    flag.Usage()
    os.Exit(2)
}
err := cmdFn(ctx, w, dataClient)
if err != nil {
    fmt.Fprintf(w, "%s failed with %v", cmd, err)
}
return err

```

完成後，`run` 函式應如下所示：

```

func run(ctx context.Context, adminClient *database.DatabaseAdminClient, dataClient
*spanner.Client, w io.Writer,
    cmd string, db string, timespan int) error {
    // createdatabase command
    if cmd == "createdatabase" {
        err := createDatabase(ctx, w, adminClient, db)
        if err != nil {
            fmt.Fprintf(w, "%s failed with %v", cmd, err)
        }
        return err
    }

    // insert and query commands
    cmdFn := commands[cmd]
    if cmdFn == nil {
        flag.Usage()
        os.Exit(2)
    }
    err := cmdFn(ctx, w, dataClient)
    if err != nil {
        fmt.Fprintf(w, "%s failed with %v", cmd, err)
    }
    return err
}

```

如要完成在應用程式中新增「插入」功能，最後一個步驟是在 `flag.Usage()` 函式中新增「insertplayers」和「insertscores」指令的說明文字。在 `flag.Usage()` 函式中加入下列說明文字，即可為插入指令加入說明文字：

將這兩個指令新增至可能的指令清單：

```

Command can be one of: createdatabase, insertplayers, insertscores

```

並在 `createdatabase` 指令的說明文字下方新增這段額外說明文字。

```

    leaderboard insertplayers projects/my-project/instances/my-
instance/databases/example-db
        - Insert 100 sample Player records into the database.
    leaderboard insertscores projects/my-project/instances/my-instance/databases/example-
db
        - Insert sample score data into Scores sample Cloud Spanner database table.

```

在 Cloud Shell 編輯器的「File」(檔案) 選單中選取「Save」(儲存)，即可儲存對 `leaderboard.go` 檔案所做的變更。

您可以使用 `golang-samples/spanner/spanner_leaderboard` 目錄中的 `leaderboard.go` 檔案，查看新增程式碼以啟用 `insertplayers` 和 `insertscores` 指令後，`leaderboard.go` 檔案的範例。

現在建構並執行應用程式，確認應用程式的可能指令清單中包含新的 `insertplayers` 和 `insertscores` 指令。執行下列指令來建構應用程式：

```

go build leaderboard.go

```

輸入下列指令，在 Cloud Shell 中執行產生的應用程式：

```
./leaderboard
```

您現在應該會在應用程式的預設輸出內容中看到 `insertplayers` 和 `insertscores` 指令：

```
Usage: leaderboard <command> <database_name> [command_option]
```

```
Command can be one of: createdatabase, insertplayers, insertscores
```

Examples:

```
leaderboard createdatabase projects/my-project/instances/my-  
instance/databases/example-db
```

- Create a sample Cloud Spanner database along with sample tables in your project.

```
leaderboard insertplayers projects/my-project/instances/my-  
instance/databases/example-db
```

- Insert 100 sample Player records into the database.

```
leaderboard insertscores projects/my-project/instances/my-  
instance/databases/example-db
```

- Insert sample score data into Scores sample Cloud Spanner database table.

現在，請使用呼叫 `createdatabase` 指令時使用的相同引數值，執行 `insertplayers` 指令。請務必將 `my-project` 替換為您在本程式碼研究室一開始建立的專案 ID。

```
./leaderboard insertplayers projects/my-project/instances/cloudspanner-  
leaderboard/databases/leaderboard
```

幾秒後，您應該會看到類似以下的回應：

```
Inserted players
```

現在讓我們使用 Go 用戶端程式庫，在 `Scores` 資料表中填入四個隨機分數，以及 `Players` 資料表中每位玩家的時間戳記。

先前執行 `create` 指令時，我們透過下列 SQL 陳述式將 `Scores` 資料表的 `Timestamp` 資料欄定義為「提交時間戳記」資料欄：

```
CREATE TABLE Scores(  
  PlayerId INT64 NOT NULL,  
  Score INT64 NOT NULL,  
  Timestamp TIMESTAMP NOT NULL OPTIONS(allow_commit_timestamp=true)  
) PRIMARY KEY(PlayerId, Timestamp),  
  INTERLEAVE IN PARENT Players ON DELETE NO ACTION
```

請注意 `OPTIONS(allow_commit_timestamp=true)` 屬性。這會將 `Timestamp` 設為「修訂時間戳記」資料欄，並在特定資料表列上執行 `INSERT` 和 `UPDATE` 作業時，自動填入確切的交易時間戳記。

您也可以將自己的時間戳記值插入「修訂時間戳記」欄，只要插入的值是過去的時間戳記即可。在本程式碼研究室中，我們將這麼做。

現在，請使用呼叫 `insertplayers` 指令時使用的相同引數值，執行 `insertscores` 指令。**請務必**將 `my-project` 替換為您在本程式碼研究室一開始建立的專案 ID。

```
./leaderboard insertscores projects/my-project/instances/cloudspanner-  
leaderboard/databases/leaderboard
```

幾秒後，您應該會看到類似以下的回應：

```
Inserted scores
```

執行 `insertScores` 函式時，會使用下列程式碼片段插入隨機產生的時間戳記，以及過去的日期時間：

```
now := time.Now()  
endDate := now.Unix()  
past := now.AddDate(0, -24, 0)  
startDate := past.Unix()  
randomDateInSeconds := rand.Int63n(endDate-startDate) + startDate  
randomDate := time.Unix(randomDateInSeconds, 0)  
stmts = append(stmts, spanner.Statement{  
    SQL: `INSERT INTO Scores  
        (PlayerId, Score, Timestamp)  
        VALUES (@playerID, @score, @timestamp)`,  
    Params: map[string]interface{}{  
        "playerID": playerID,  
        "score":     score,  
        "timestamp": randomDate,  
    },  
})
```

若要自動在 `Timestamp` 欄中填入「插入」交易發生的確切時間戳記，可以改為插入 Go 常數 `spanner.CommitTimestamp`，如下列程式碼片段所示：

```
...  
stmts = append(stmts, spanner.Statement{  
    SQL: `INSERT INTO Scores  
        (PlayerId, Score, Timestamp)  
        VALUES (@playerID, @score, @timestamp)`,  
    Params: map[string]interface{}{  
        "playerID": playerID,  
        "score":     score,  
        "timestamp": spanner.CommitTimestamp,  
    },  
})
```


資料載入作業已完成，現在請前往 Cloud Console 的 [Cloud Spanner 專區](#)，驗證我們剛才寫入新資料表的值。首先選取「`leaderboard`」資料庫，然後選取「`Players`」資料表。按一下「`Data`」分頁標籤。您應該會看到資料表 `PlayerId` 和 `PlayerName` 欄中的資料。

Spanner

TABLE

Schema

Indexes

Data

Query

All instances > INSTANCE cloudspanner-leaderboard: Overview > DATABASE leaderboard: Overview > TABLE Players: Data

Data

INSERT

EDIT

DELETE

Filter

Filter data

☐	PlayerId 	PlayerName
☐	1006673974	Player 28
☐	1157007144	Player 57
☐	1282007052	Player 44
☐	1307959065	Player 5
☐	1426930072	Player 7
☐	1427121301	Player 40
☐	1461854546	Player 91
☐	1523212851	Player 82
☐	1545939357	Player 96

接著，按一下「`Scores`」資料表並選取「`Data`」分頁標籤，確認「`Scores`」資料表也有資料。您應該會看到資料表中的 `PlayerId`、`Timestamp` 和 `Score` 欄位有資料。



Spanner

TABLE



Schema



Indexes



Data



Query

All instances > INSTANCE cloudspanner-leaderboard: Overview > DATABASE leaderboard: Overview > TABLE Scores: Data

Data

INSERT

EDIT

DELETE

 Filter

Filter data

☐	PlayerId 	Timestamp 	Score
☐	1006673974	2019-06-16T14:32:56Z	164829
☐	1006673974	2019-07-04T01:39:47Z	195219
☐	1006673974	2019-07-30T18:19:30Z	723686
☐	1006673974	2019-09-30T18:14:29Z	717320
☐	1157007144	2019-06-04T15:51:32Z	986626
☐	1157007144	2019-10-05T02:53:18Z	408715
☐	1157007144	2020-07-27T17:30:29Z	19131
☐	1157007144	2020-12-16T01:46:36Z	770912
☐	1282007052	2019-09-21T23:26:59Z	219855
☐	1282007052	2020-09-26T12:56:09Z	875433

非常好！現在來更新應用程式，執行一些可用於建立遊戲排行榜的查詢。

6. 執行排行榜查詢

我們已設定資料庫並將資訊載入資料表，現在要使用這些資料建立排行榜。為此，我們需要回答下列四個問題：

1. 哪幾位球員是史上「前十名」？
2. 哪些球員是年度「十大」球員？
3. 哪些玩家是當月的「前十名」？
4. 本週「前十名」的播放器是哪些？

現在來更新應用程式，執行可回答這些問題的 SQL 查詢。

我們會新增 `query` 指令和 `queryWithTimespan` 指令，方便您執行查詢，回答問題並產生排行榜所需的資訊。

在 Cloud Shell 編輯器中編輯 `leaderboard.go` 檔案，更新應用程式以新增 `query` 指令和 `queryWithTimespan` 指令。我們也會新增 `formatWithCommas` 輔助函式，以逗號分隔分數。

首先，請更新 `leaderboard.go` 檔案頂端的 `imports` 區段，取代目前內容，完成後應如下所示：

```
import (  
    "bytes"  
    "context"  
    "flag"  
    "fmt"  
    "io"  
    "log"  
    "math/rand"  
    "os"  
    "regexp"  
    "strconv"  
    "time"  
  
    "cloud.google.com/go/spanner"  
    database "cloud.google.com/go/spanner/admin/database/apiv1"  
  
    "google.golang.org/api/iterator"  
    adminpb  "google.golang.org/genproto/googleapis/spanner/admin/database/v1"  
)
```

接著，在現有的 `insertScores` 方法下方新增下列兩個函式和輔助函式：

```

func query(ctx context.Context, w io.Writer, client *spanner.Client) error {
    stmt := spanner.Statement{
        SQL: `SELECT p.PlayerId, p.PlayerName, s.Score, s.Timestamp
            FROM Players p
            JOIN Scores s ON p.PlayerId = s.PlayerId
            ORDER BY s.Score DESC LIMIT 10`
    }
    iter := client.Single().Query(ctx, stmt)
    defer iter.Stop()
    for {
        row, err := iter.Next()
        if err == iterator.Done {
            return nil
        }
        if err != nil {
            return err
        }
        var playerId, score int64
        var playerName string
        var timestamp time.Time
        if err := row.Columns(&playerId, &playerName, &score, &timestamp); err != nil
    {
        return err
    }
    fmt.Fprintf(w, "PlayerId: %d PlayerName: %s Score: %s Timestamp: %s\n",
        playerId, playerName, formatWithCommas(score), timestamp.String()
[0:10])
    }
}

func queryWithTimespan(ctx context.Context, w io.Writer, client *spanner.Client, timespan
int) error {
    stmt := spanner.Statement{
        SQL: `SELECT p.PlayerId, p.PlayerName, s.Score, s.Timestamp
            FROM Players p
            JOIN Scores s ON p.PlayerId = s.PlayerId
            WHERE s.Timestamp > TIMESTAMP_SUB(CURRENT_TIMESTAMP(),
INTERVAL @Timespan HOUR)
            ORDER BY s.Score DESC LIMIT 10`,
        Params: map[string]interface{}{"Timespan": timespan},
    }
    iter := client.Single().Query(ctx, stmt)
    defer iter.Stop()
    for {
        row, err := iter.Next()
        if err == iterator.Done {
            return nil
        }
        if err != nil {
            return err
        }
        var playerId, score int64
        var playerName string

```

```

        var timestamp time.Time
        if err := row.Columns(&playerID, &playerName, &score, &timestamp); err != nil
        {
            return err
        }
        fmt.Fprintf(w, "PlayerId: %d  PlayerName: %s  Score: %s  Timestamp: %s\n",
            playerID, playerName, formatWithCommas(score), timestamp.String()
[0:10])
    }
}

func formatWithCommas(n int64) string {
    numberAsString := strconv.FormatInt(n, 10)
    numberLength := len(numberAsString)
    if numberLength < 4 {
        return numberAsString
    }
    var buffer bytes.Buffer
    comma := []rune(",")
    bufferPosition := numberLength % 3
    if (bufferPosition) > 0 {
        bufferPosition = 3 - bufferPosition
    }
    for i := 0; i < numberLength; i++ {
        if bufferPosition == 3 {
            buffer.WriteRune(comma[0])
            bufferPosition = 0
        }
        bufferPosition++
        buffer.WriteByte(numberAsString[i])
    }
    return buffer.String()
}

```

接著在 `leaderboard.go` 檔案頂端，於 `commands` 變數中新增「query」做為其中一個指令選項，位置就在「`insertscores`」選項下方，讓 `commands` 變數看起來像這樣：

```

var (
    commands = map[string]command{
        "insertplayers": insertPlayers,
        "insertscores":  insertScores,
        "query":         query,
    }
)

```

接著，在 `run` 函式中新增「`queryWithTimespan`」做為指令選項，位置在「`createdatabase`」指令區段下方，以及「`insert and query`」指令處理區段上方：

```

// querywithtimespan command
if cmd == "querywithtimespan" {
    err := queryWithTimespan(ctx, w, dataClient, timespan)
    if err != nil {
        fmt.Fprintf(w, "%s failed with %v", cmd, err)
    }
    return err
}

```

完成後，`run` 函式應如下所示：

```

func run(ctx context.Context, adminClient *database.DatabaseAdminClient, dataClient
*spanner.Client, w io.Writer,
    cmd string, db string, timespan int) error {
    // createdatabase command
    if cmd == "createdatabase" {
        err := createDatabase(ctx, w, adminClient, db)
        if err != nil {
            fmt.Fprintf(w, "%s failed with %v", cmd, err)
        }
        return err
    }

    // querywithtimespan command
    if cmd == "querywithtimespan" {
        if timespan == 0 {
            flag.Usage()
            os.Exit(2)
        }
        err := queryWithTimespan(ctx, w, dataClient, timespan)
        if err != nil {
            fmt.Fprintf(w, "%s failed with %v", cmd, err)
        }
        return err
    }

    // insert and query commands
    cmdFn := commands[cmd]
    if cmdFn == nil {
        flag.Usage()
        os.Exit(2)
    }
    err := cmdFn(ctx, w, dataClient)
    if err != nil {
        fmt.Fprintf(w, "%s failed with %v", cmd, err)
    }
    return err
}

```

接著，如要讓 `queryWithTimespan` 指令正常運作，請更新應用程式「main」方法中的 `flag.Parse()` 程式碼區塊，如下所示：

```
flag.Parse()
flagCount := len(flag.Args())
if flagCount < 2 || flagCount > 3 {
    flag.Usage()
    os.Exit(2)
}

cmd, db := flag.Arg(0), flag.Arg(1)
// If query timespan flag is specified, parse to int
var timespan int = 0
if flagCount == 3 {
    parsedTimespan, err := strconv.Atoi(flag.Arg(2))
    if err != nil {
        fmt.Println(err)
        os.Exit(2)
    }
    timespan = parsedTimespan
}

ctx, cancel := context.WithTimeout(context.Background(), 1*time.Minute)
defer cancel()
adminClient, dataClient := createClients(ctx, db)
if err := run(ctx, adminClient, dataClient, os.Stdout, cmd, db, timespan); err != nil
{
    os.Exit(1)
}
```

如要完成在應用程式中新增「查詢」功能，最後一個步驟是在 `flag.Usage()` 函式中，為「query」和「querywithtimespan」指令新增說明文字。在 `flag.Usage()` 函式中加入下列程式碼行，即可納入查詢指令的說明文字：

將兩個「query」指令新增至可能的指令清單：

```
Command can be one of: createdatabase, insertplayers, insertscores, query, querywithtimespan
```

並在 `insertscores` 指令的說明文字下方新增這段額外說明文字。

```
leaderboard query projects/my-project/instances/my-instance/databases/example-db
    - Query players with top ten scores of all time.
leaderboard querywithtimespan projects/my-project/instances/my-
instance/databases/example-db 168
    - Query players with top ten scores within a timespan specified in hours.
```

在 Cloud Shell 編輯器的「File」（檔案）選單中選取「Save」（儲存），即可儲存對 `leaderboard.go` 檔案所做的變更。

您可以使用 `golang-samples/spanner/spanner_leaderboard` 目錄中的 `leaderboard.go` 檔案，查看新增程式碼以啟用 `query` 和 `querywithtimespan` 指令後，`leaderboard.go` 檔案的範例。

現在建構並執行應用程式，確認新的 `query` 和 `querywithtimespan` 指令已納入應用程式的可能指令清單。

在 Cloud Shell 中執行下列指令，建構應用程式：

```
go build leaderboard.go
```

輸入下列指令，在 Cloud Shell 中執行產生的應用程式：

```
./leaderboard
```

您現在應該會在應用程式的預設輸出內容中看到 `query` 和 `querywithtimespan` 指令，做為新的指令選項：

```
Usage: leaderboard <command> <database_name> [command_option]
```

```
        Command can be one of: createdatabase, insertplayers, insertscores, query,
querywithtimespan
```

Examples:

```
    leaderboard createdatabase projects/my-project/instances/my-
instance/databases/example-db
```

```
        - Create a sample Cloud Spanner database along with sample tables in
your project.
```

```
    leaderboard insertplayers projects/my-project/instances/my-
instance/databases/example-db
```

```
        - Insert 100 sample Player records into the database.
```

```
    leaderboard insertscores projects/my-project/instances/my-
instance/databases/example-db
```

```
        - Insert sample score data into Scores sample Cloud Spanner database
table.
```

```
    leaderboard query projects/my-project/instances/my-
instance/databases/example-db
```

```
        - Query players with top ten scores of all time.
```

```
    leaderboard querywithtimespan projects/my-project/instances/my-
instance/databases/example-db 168
```

```
        - Query players with top ten scores within a timespan specified in
hours.
```

從回覆中可以看出，我們可以使用 `query` 指令，取得歷來「十大」球員的清單。我們也可以看到，`querywithtimespan` 指令可讓我們指定時間範圍 (以小時為單位)，根據 `Scores` 表格 `Timestamp` 欄中的值篩選記錄。

請使用執行 `create` 指令時所用的相同引數值，執行 `query` 指令。**請務必**將 `my-project` 替換為您在本程式碼研究室一開始建立的專案 ID。

```
./leaderboard query projects/my-project/instances/cloudspanner-
leaderboard/databases/leaderboard
```

畫面會顯示類似以下的回應，列出歷來排名前十的球員：

```

PlayerId: 4018687297 PlayerName: Player 83 Score: 999,618 Timestamp: 2017-07-01
PlayerId: 4018687297 PlayerName: Player 83 Score: 998,956 Timestamp: 2017-09-02
PlayerId: 4285713246 PlayerName: Player 51 Score: 998,648 Timestamp: 2017-12-01
PlayerId: 5267931774 PlayerName: Player 49 Score: 997,733 Timestamp: 2017-11-09
PlayerId: 1981654448 PlayerName: Player 35 Score: 997,480 Timestamp: 2018-12-06
PlayerId: 4953940705 PlayerName: Player 87 Score: 995,184 Timestamp: 2018-09-14
PlayerId: 2456736905 PlayerName: Player 84 Score: 992,881 Timestamp: 2017-04-14
PlayerId: 8234617611 PlayerName: Player 19 Score: 992,399 Timestamp: 2017-12-27
PlayerId: 1788051688 PlayerName: Player 76 Score: 992,265 Timestamp: 2018-11-22
PlayerId: 7127686505 PlayerName: Player 97 Score: 992,038 Timestamp: 2017-12-02

```

現在，請使用必要引數執行 `querywithtimespan` 指令，指定「timespan」等於一年內的時數 (8760)，查詢該年度的「前十名」玩家。**請務必**將 `my-project` 替換為您在本程式碼研究室一開始建立的專案 ID。

```

./leaderboard querywithtimespan projects/my-project/instances/cloudspanner-
leaderboard/databases/leaderboard 8760

```

畫面會顯示類似以下的回應，其中包含年度「前十名」玩家：

```

PlayerId: 1981654448 PlayerName: Player 35 Score: 997,480 Timestamp: 2018-12-06
PlayerId: 4953940705 PlayerName: Player 87 Score: 995,184 Timestamp: 2018-09-14
PlayerId: 1788051688 PlayerName: Player 76 Score: 992,265 Timestamp: 2018-11-22
PlayerId: 6862349579 PlayerName: Player 30 Score: 990,877 Timestamp: 2018-09-14
PlayerId: 5529627211 PlayerName: Player 16 Score: 989,142 Timestamp: 2018-03-30
PlayerId: 9743904155 PlayerName: Player 1 Score: 988,765 Timestamp: 2018-05-30
PlayerId: 6809119884 PlayerName: Player 7 Score: 986,673 Timestamp: 2018-05-16
PlayerId: 2132710638 PlayerName: Player 54 Score: 983,108 Timestamp: 2018-09-11
PlayerId: 2320093590 PlayerName: Player 79 Score: 981,373 Timestamp: 2018-05-07
PlayerId: 9554181430 PlayerName: Player 80 Score: 981,087 Timestamp: 2018-06-21

```

注意：由於應用程式的 `insertscores` 指令會插入過去 2 年隨機產生的時間戳記分數，因此 `querywithtimespan` 指令的結果可能不會列出 10 位玩家，因為這些分數可能落在您指定的時間範圍內，也可能不在。您可以重新執行 `insertscores` 指令，插入更多分數：

```

./leaderboard insertscores projects/my-project/instances/cloudspanner-
leaderboard/databases/leaderboard

```

接著重新執行 `querywithtimespan` 指令，看看結果是否會發生變化。

現在執行 `querywithtimespan` 指令，指定「timespan」等於一個月的小時數 (730)，查詢當月「前十名」玩家。**請務必**將 `my-project` 替換為您在本程式碼研究室一開始建立的專案 ID。

```

./leaderboard querywithtimespan projects/my-project/instances/cloudspanner-
leaderboard/databases/leaderboard 730

```

畫面會顯示當月「前十名」玩家，如下所示：


```
PlayerId: 3869829195 PlayerName: Player 69 Score: 949,686 Timestamp: 2019-02-19
PlayerId: 7448359883 PlayerName: Player 20 Score: 938,998 Timestamp: 2019-02-07
PlayerId: 1981654448 PlayerName: Player 35 Score: 929,003 Timestamp: 2019-02-22
PlayerId: 9336678658 PlayerName: Player 44 Score: 914,106 Timestamp: 2019-01-27
PlayerId: 6968576389 PlayerName: Player 40 Score: 898,041 Timestamp: 2019-02-21
PlayerId: 5529627211 PlayerName: Player 16 Score: 896,433 Timestamp: 2019-01-29
PlayerId: 9395039625 PlayerName: Player 59 Score: 879,495 Timestamp: 2019-02-09
PlayerId: 2094604854 PlayerName: Player 39 Score: 860,434 Timestamp: 2019-02-01
PlayerId: 9395039625 PlayerName: Player 59 Score: 849,955 Timestamp: 2019-02-21
PlayerId: 4285713246 PlayerName: Player 51 Score: 805,654 Timestamp: 2019-02-02
```

現在，請執行 `querywithtimespan` 指令，指定「timespan」等於一週的小時數 (168)，查詢當週「前十名」玩家。**請務必將 `my-project` 替換為您在本程式碼研究室一開始建立的專案 ID。**

```
./leaderboard querywithtimespan projects/my-project/instances/cloudspanner-
leaderboard/databases/leaderboard 168
```

畫面會顯示本週「前十名」玩家，如下所示：

```
PlayerId: 3869829195 PlayerName: Player 69 Score: 949,686 Timestamp: 2019-02-19
PlayerId: 1981654448 PlayerName: Player 35 Score: 929,003 Timestamp: 2019-02-22
PlayerId: 6968576389 PlayerName: Player 40 Score: 898,041 Timestamp: 2019-02-21
PlayerId: 9395039625 PlayerName: Player 59 Score: 849,955 Timestamp: 2019-02-21
PlayerId: 5954045812 PlayerName: Player 8 Score: 795,639 Timestamp: 2019-02-22
PlayerId: 3889939638 PlayerName: Player 71 Score: 775,252 Timestamp: 2019-02-21
PlayerId: 5529627211 PlayerName: Player 16 Score: 604,695 Timestamp: 2019-02-19
PlayerId: 9006728426 PlayerName: Player 3 Score: 457,208 Timestamp: 2019-02-22
PlayerId: 8289497066 PlayerName: Player 58 Score: 227,697 Timestamp: 2019-02-20
PlayerId: 8065482904 PlayerName: Player 99 Score: 198,429 Timestamp: 2019-02-24
```

太棒了！

現在新增記錄時，Cloud Spanner 會將資料庫擴充至所需大小。無論資料庫成長多少，遊戲排行榜都能透過 Cloud Spanner 及其 Truetime 技術，持續準確地擴充。

步驟 7 · 約 2 分鐘

7. 清除所用資源

盡情體驗 Spanner 的樂趣後，我們需要清理遊樂場，節省寶貴的資源和金錢。幸好這個步驟很簡單，只要前往 Cloud 控制台的「Cloud Spanner」專區，然後刪除在 Codelab 步驟「設定 Cloud Spanner 執行個體」中建立的執行個體即可。

8. 恭喜！

涵蓋內容：

- 排行榜的 Google Cloud Spanner 執行個體、資料庫和資料表結構
- 如何建立 Go 控制台應用程式
- 如何使用 Go 用戶端程式庫建立 Spanner 資料庫和資料表
- 如何使用 Go 用戶端程式庫將資料載入 Spanner 資料庫
- 如何使用 Spanner 提交時間戳記和 Go 用戶端程式庫，從資料中查詢「前十名」結果

後續步驟：

- 閱讀 [Spanner CAP 白皮書](#)
- 瞭解[結構定義設計](#)和查詢最佳做法
- 進一步瞭解 Cloud Spanner [提交時間戳記](#)

提供意見

- 請花點時間填寫這份簡短的[問卷調查](#)
-